

Week1 Git版本控制以及LaTeX文档编辑

赵均临

August 2026

目录

1	课上实验题目	1
1.1	实验1: 含空格文件名的批量整理	1
1.2	实验2: 随机访问日志统计	3
1.2.1	操作过程与命令解析	3
1.2.2	运行测试与结果验证	4
1.3	实验3: 制造、解决并解释一次合并冲突	5
1.3.1	操作过程与命令解析	5
1.3.2	运行测试与结果验证	6
1.4	实验4: 修复并构建一页技术说明	7
1.4.1	操作过程与命令解析	7
1.4.2	运行测试与结果验证	8
2	练习: Shell入门	10
2.1	ls的作用	10
2.2	三种引号的区别与特殊字符输出	10
2.3	为什么cd必须是Shell内建命令	11
2.4	脚本赋权与执行	12
2.5	命令替换与日期备份	12
2.6	使用awk按列过滤与交换字段	12
2.7	克隆本课程网站的仓库	13

2.7.1	可视化版本历史为图	13
2.7.2	查看最后修改README.md的人	13
2.8	配置Git别名	14
2.9	配置全局.gitignore	14
2.10	模拟合并冲突+解决	14
3	实验心得	17

github网址: <https://github.com/coolicer17/Systemtools>

1 课上实验题目

1.1 实验1: 含空格文件名的批量整理

1. 实验目的与内容(摘要)

掌握 Shell 基础文件操作, 包括含空格/隐藏文件创建、绝对路径查看、使用 rsync 批量复制并保留目录结构、权限管理 (chmod 750/640), 以及利用 find+stat 生成文件清单。

2. 操作步骤以及指令

步骤1: 创建目录与文件

```
mkdir -p q01
cd q01
mkdir -p input/docs input/tmp
printf "alpha\nbeta\n" > input/docs/"notes one.txt"
echo "hidden" > input/docs/.secret.txt
touch input/tmp/empty.txt
echo "system start> running task ok" > input/run.log
```

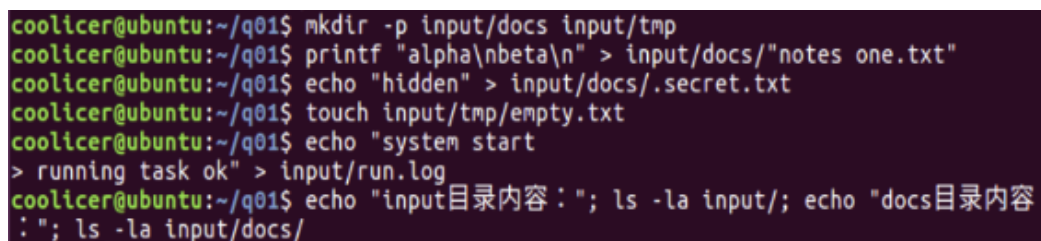
-p代表父目录, 如果上级目录不存在能同时创建开始代码块

用printf实现多行换行, 防止echo直接输出换行符

Linux下文件以.开头默认隐藏

touch创建空文件

echo输出两行文字”system start”和”running task ok”



```
coolicer@ubuntu:~/q01$ mkdir -p input/docs input/tmp
coolicer@ubuntu:~/q01$ printf "alpha\nbeta\n" > input/docs/"notes one.txt"
coolicer@ubuntu:~/q01$ echo "hidden" > input/docs/.secret.txt
coolicer@ubuntu:~/q01$ touch input/tmp/empty.txt
coolicer@ubuntu:~/q01$ echo "system start
> running task ok" > input/run.log
coolicer@ubuntu:~/q01$ echo "input目录内容:"; ls -la input/; echo "docs目录内容:"; ls -la input/docs/
```

步骤2:查看路径与列表

echo "q01绝对路径:" && pwd ——pwd:输出当前绝对路径

输出: /home/coolicer/q01

ls -la input/ ——-l长格式, -a显示隐藏文件

含隐藏文件, 确认 docs、tmp、run.log

步骤3: 复制txt文件(保留结构, 处理空格)

STUID="2502000718" ——-定义学号变量, 后续\$STUID调用

mkdir -p work/\$STUID ——-创建目标目录

rsync -avm --include='*.txt' --include='*/' --exclude='*' input/ work/\$STUID/

--include='*.txt': 包含全部 txt 文件(匹配含空格的 notes one.txt)

--include='*/': 包含所有目录(确保相对路径结构)

--exclude='*': 排除所有其他内容(如 run.log 非 txt, 被排除)

rsync用于复制, work/\$STUID/为目标路径

```
coolicer@ubuntu:~/q01$ STUID="2502000718"
coolicer@ubuntu:~/q01$ mkdir -p work/$STUID
coolicer@ubuntu:~/q01$ rsync -avm --include='*.txt' --include='*/' --exclude='*'
input/ work/$STUID/
```

步骤4: 权限设置于清单生成

```
find work/$STUID -type d -exec chmod 750 {}
```

```
find work/$STUID -type f -exec chmod 640 {} +
```

```
cd work/$STUID && find . -type f -exec stat -c '%n\s' {} \; > ../../inventory.txt
```

```
cat ../../inventory.txt
```

4. 实验结果验证

目录结构: work/2502000718/input/docs,tmp 下仅含 .txt 文件, 含空格文件 notes one.txt 完整复制。

权限: 目录 750 (drwxr-x—), 文件 640 (-rw-r—)。

inventory.txt 内容:

纯文本

./docs/.secret.txt 7

./docs/notes one.txt 11

./tmp/empty.txt 0

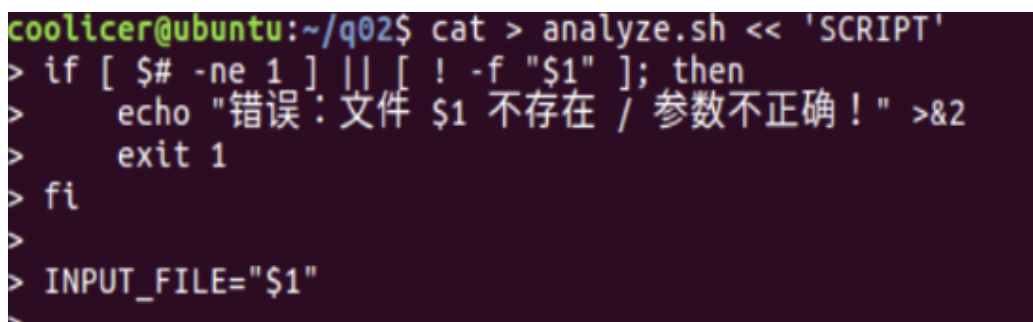
符合“相对路径+字节数”的要求

1.2 实验2：随机访问日志统计

1.2.1 操作过程与命令解析

步骤1. 目录创建与数据准备

在 q02 目录下，通过 `cat` 以 EOF 创建 `access.csv`，包含 1 行表头与 8 行访问日志。为后续统计提供原始数据。



```
coolicer@ubuntu:~/q02$ cat > analyze.sh << 'SCRIPT'
> if [ $# -ne 1 ] || [ ! -f "$1" ]; then
>     echo "错误：文件 $1 不存在 / 参数不正确！" >&2
>     exit 1
> fi
>
> INPUT_FILE="$1"
```

步骤2. 编写 `analyze.sh` 与参数校验

通过 `cat > analyze.sh << 'SCRIPT'` 写入脚本，核心逻辑如下：

- **参数校验：** `if [ $# -ne 1 ] || [ ! -f "$1" ]` 判断参数数量是否为 1 且文件是否存在；若异常则 `echo "错误..." >&2`（重定向到标准错误 `stderr`），并执行 `exit 1` 返回非零退出码。
 - **变量赋值：** `INPUT_FILE="$1"` 记录文件路径。
- SCRIPT:结束脚本写入

步骤3. HTTP 5xx 统计

- **5xx 统计：** `tail -n +2` 跳过表头 → `awk -F',' '$4 >= 500 {print $3}'` 以逗号为分隔符，筛选第4列（status） ≥ 500 （5xx），输出第3列（path）→ `sort | uniq -c` 计数 → `sort -k1nr -k2d` 按次数降序、路径字典序排 → `head -n2` 取前 2 → `awk '{printf "%s : %d次\n", $2, $1}'` 格式化输出。

```

> echo "=== HTTP 5xx 最多的前2个接口路径 ==="
> tail -n +2 "$INPUT_FILE" | awk -F',' ' $4 >= 500 {print $3}' | sort | uniq -c |
sort -k1nr -k2d | head -n2 | awk '{printf "%s : %d次\n", $2, $1}'
>
> echo ""
> echo "=== 所有请求平均延迟 latency_ms ==="
> tail -n +2 "$INPUT_FILE" | awk -F',' '{sum += $5; cnt++} END {printf "%.2f ms\n", sum/cnt}'
> SCRIPT

```

步骤4.平均延迟时间计算

- 平均延迟: `tail -n +2` 跳过表头 → `awk -F',' '{sum += $5; cnt++} END {printf "%.2f ms\n", sum/cnt}'` 累加第 5 列并求均值, 保留两位。`%.2f`: 强制保留两位小数
- `tail`: 默认用于查看文件末尾内容
- `"$INPUT_FILE"`: 传入的CSV文件路径。加双引号可避免路径含空格时出错

写入后执行 `chmod +x analyze.sh` 赋予可执行权限。

```

coolicer@ubuntu:~/q02$ ./analyze.sh access.csv
=== HTTP 5xx 最多的前2个接口路径 ===
/api/orders : 3次
/api/users : 2次

```

1.2.2 运行测试与结果验证

- 正常执行: `./analyze.sh access.csv` 输出:

```

=== HTTP 5xx 最多的前2个接口路径 ===
/api/orders : 3次
/api/users : 2次
=== 所有请求平均延迟 latency_ms ===
193.75 ms

```

结果符合数据特征（orders 出现 3 次 5xx，users 2 次；总延迟 1550ms/8条=193.75ms）。

- 异常执行：./analyze.sh 不存在的文件.csv 输出 错误：文件不存在的文件.csv 不存在 / 参数不正确！ 到 stderr，随后 echo "脚本退出码：\$?" 显示 1，符合非零退出要求。

1.3 实验3：制造、解决并解释一次合并冲突

1.3.1 操作过程与命令解析

- 初始化与配置：执行 git init 初始化仓库。通过 git config 设置用户身份。使用 echo 重定向写入 config.txt 并 git commit 完成初始提交。执行 git branch -M main 设定主分支。

```
coolicer@ubuntu:~/q03$ git add config.txt
coolicer@ubuntu:~/q03$ git commit -m "init: add config.txt with mode=normal"
[master (root-commit) af3b2db] init: add config.txt with mode=normal
1 file changed, 1 insertion(+)
create mode 100644 config.txt
coolicer@ubuntu:~/q03$ git branch -M main
coolicer@ubuntu:~/q03$ git checkout -b feature-a # 基于当前main创建并切换到feature-a
```

- 分支开发与冲突制造：使用 git checkout -b feature-a 创建特性分支并修改为 mode=safe 提交；切回 main 后，基于初始状态创建 feature-b 并修改为 mode=fast 提交。两分支对同一文件行做了不同修改。

```
coolicer@ubuntu:~/q03$ git merge feature-a
Updating af3b2db..fab293
Fast-forward
 config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
coolicer@ubuntu:~/q03$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
coolicer@ubuntu:~/q03$ cat config.txt
```

- 合并与冲突解决：切回 main，git merge feature-a 提示 Fast-forward 成功。合并 feature-b 时提示 CONFLICT (content)。通过 cat 查看冲突标记 (<<<<<< HEAD 等)。使用 cat > config.txt << 'EOF' 覆盖写入最终内容（保留 safe 加 note=reviewed）。执行 git add 标记已解决，并提交合并信息。

```

oolicer@ubuntu:~/q03$ git merge feature-a
Updating af3b2db..fabcb293
Fast-forward
 config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
oolicer@ubuntu:~/q03$ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
oolicer@ubuntu:~/q03$ cat config.txt

```

图 1: Git 合并历史拓扑图验证

1.3.2 运行测试与结果验证

- 状态与历史验证：执行 git status 确认工作区干净（working directory clean）。
- 拓扑图验证：执行 git log --all --graph --decorate --oneline，终端输出图形化历史，确认 feature-a 和 feature-b 的节点已合并入 main，符合题目要求的分支拓扑结构。

```

oolicer@ubuntu:~/q03$ # 直接写入最终内容（覆盖冲突标记）
oolicer@ubuntu:~/q03$ cat > config.txt << 'EOF'
> mode=safe
> note=reviewed
> EOF
oolicer@ubuntu:~/q03$ git add config.txt # 标记冲突已解决
oolicer@ubuntu:~/q03$ git commit -m "resolve merge: keep safe mode + add reviewed note"

```

图 2: Git 结果验证

1.4 实验4：修复并构建一页技术说明

1.4.1 操作过程与命令解析

- **目录创建与文档编写：**执行 `mkdir -p q04` 创建实验目录并进入。使用 `nano report.tex` 创建并编辑 LaTeX 源码文件，补全文档类、宏包、公式、表格和交叉引用代码。

```
coolicer@ubuntu:~$ cd q04
coolicer@ubuntu:~/q04$ nano report.tex
coolicer@ubuntu:~/q04$ latexmk -pdf -halt-on-error report.tex
```

图 3: 创建 q04 目录并编辑 report.tex

- **文档结构与公式：**使用 `\documentclass{article}` 初始化文档，引入 `amsmath` 宏包支持数学公式与 `\eqref` 引用。
引入公式：`\begin{equation}`
结束公式：`\end{equation}`
- **表格与标签：**使用 `table` 浮动环境和 `tabular` 环境绘制 2 行 2 列表格，通过 `\caption{系统性能参数}` 添加标题，并在其后紧跟 `\label{tab:perf}`。
注意：`label` 必须放在 `caption` 之后。
- **交叉引用：**
`\eqref{eq:throughput}` 引用公式编号并自动加括号，输出为“(1)”；
`\ref{tab:env}` 输出表格编号“1”
- **命令行构建：**在终端执行 `latexmk -pdf -halt-on-error report.tex`（或 `tectonic report.tex`）编译生成 PDF。

```

\section{Result}
The core formula for average throughput calculation is:
\begin{equation}
Throughput = \frac{Total\ Requests}{Total\ Time} \label{eq:throughput}
\end{equation}
As shown in Equation~\eqref{eq:throughput}, we calculate the QPS.

\begin{table}[h]
\centering
\begin{tabular}{|c|c|}
\hline
Config Item      & Value \\ \hline
CPU Cores        & 8      \\ \hline
\end{tabular}
\caption{Test Environment Info} \label{tab:env}
\end{table}

```

图 4: LaTeX 编译生成 PDF 部分代码展示

1.4.2 运行测试与结果验证

- **PDF 生成验证:** 编译成功后当前目录会出现 report.pdf，使用 evince report.pdf & 打开查看。
- **交叉引用验证:** 由于 LaTeX 需要多遍编译，首次编译引用处可能显示 ??。连续编译两次后，确认公式引用显示为 (1)，表格引用显示为 1，不再出现 ??。

Tool Report

Zhaojunlin - 25020007178

August 30, 2026

1 Result

The core formula for average throughput calculation is:

$$Throughput = \frac{Total\ Requests}{Total\ Time} \quad (1)$$

As shown in Equation (1), we calculate the QPS.

Config Item	Value
CPU Cores	8

Table 1: Test Environment Info

All test runs use the configuration listed in Table 1.

2 Conclusion

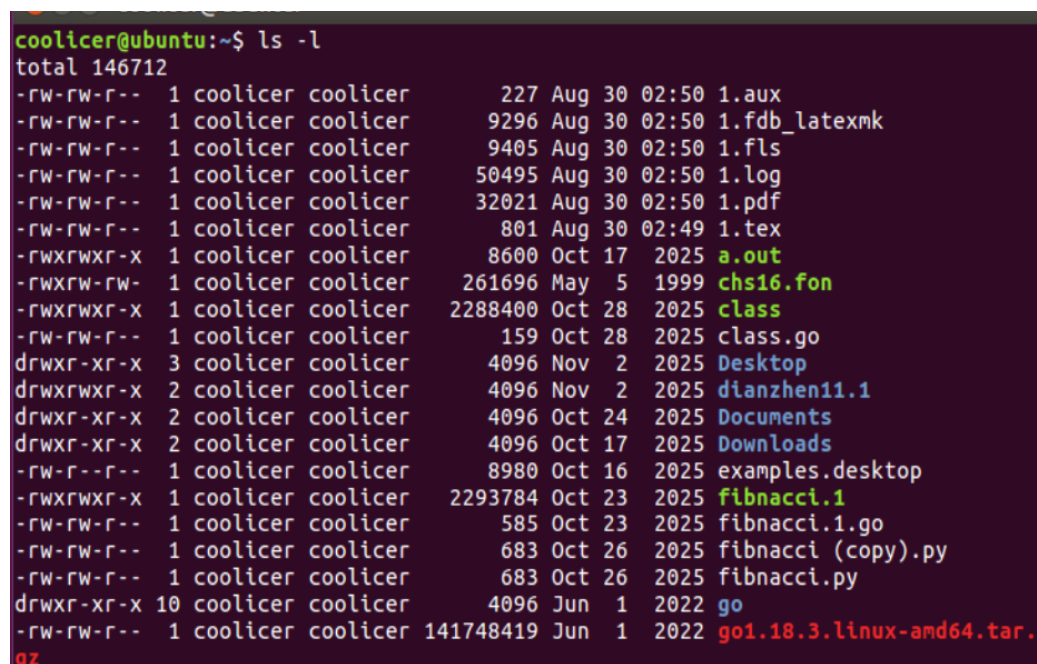
The test completes successfully.

图 5: PDF 预览与交叉引用结果验证

2 练习：Shell入门

2.1 ls的作用

ls -l 的作用是以长格式（long format）显示文件详细信息。与直接输入 ls 仅显示文件名不同，ls -l 会显示文件的权限、硬链接数、所有者、所属组、文件大小、最后修改时间和文件名。



```
coolicer@ubuntu:~$ ls -l
total 146712
-rw-rw-r-- 1 coolicer coolicer    227 Aug 30 02:50 1.aux
-rw-rw-r-- 1 coolicer coolicer   9296 Aug 30 02:50 1.fdb_latexmk
-rw-rw-r-- 1 coolicer coolicer   9405 Aug 30 02:50 1.flx
-rw-rw-r-- 1 coolicer coolicer  50495 Aug 30 02:50 1.log
-rw-rw-r-- 1 coolicer coolicer  32021 Aug 30 02:50 1.pdf
-rw-rw-r-- 1 coolicer coolicer    801 Aug 30 02:49 1.tex
-rwxrwxr-x 1 coolicer coolicer   8600 Oct 17  2025 a.out
-rwxrwxr-x 1 coolicer coolicer  261696 May  5  1999 chs16.fon
-rwxrwxr-x 1 coolicer coolicer 2288400 Oct 28  2025 class
-rw-rw-r-- 1 coolicer coolicer    159 Oct 28  2025 class.go
drwxr-xr-x 3 coolicer coolicer   4096 Nov  2  2025 Desktop
drwxrwxr-x 2 coolicer coolicer   4096 Nov  2  2025 dianzhen11.1
drwxr-xr-x 2 coolicer coolicer   4096 Oct 24  2025 Documents
drwxr-xr-x 2 coolicer coolicer   4096 Oct 17  2025 Downloads
-rw-r--r-- 1 coolicer coolicer   8980 Oct 16  2025 examples.desktop
-rwxrwxr-x 1 coolicer coolicer 2293784 Oct 23  2025 fibnacci.1
-rw-rw-r-- 1 coolicer coolicer    585 Oct 23  2025 fibnacci.1.go
-rw-rw-r-- 1 coolicer coolicer    683 Oct 26  2025 fibnacci (copy).py
-rw-rw-r-- 1 coolicer coolicer    683 Oct 26  2025 fibnacci.py
drwxr-xr-x 10 coolicer coolicer   4096 Jun  1  2022 go
-rw-rw-r-- 1 coolicer coolicer 141748419 Jun  1  2022 go1.18.3.linux-amd64.tar.gz
```

2.2 三种引号的区别与特殊字符输出

- 单引号”：强引用，内部所有字符（包括 \$、反引号、转义符）都被原样保留，不解析。
- 双引号””：弱引用，允许解析 \$（变量）、反引号（命令替换）、‘（转义）和！（历史扩展）。
- ANSI-C 引号 \$’ ’：支持解析反斜杠转义序列（如 \n 换行、\a 响铃）。

位次	常见取值	含义
1	d/-/l/b/c/a/p	文件类型
2	r/-	所有者读权限
3	w/-	所有者写权限
4	x/-/s	所有者执行权限（s=setuid）
5	r/-	所属组读权限
6	w/-	所属组写权限
7	x/-/s	所属组执行权限（s=setgid）
8	r/-	其他用户读权限
9	w/-	其他用户写权限
10	x/-/t	其他用户执行权限（t=粘滞位）

表 1: ls -l 输出前 10 个字符的含义

```

coolicer@ubuntu:~$ echo '$Line 1\n$var!\nLine 3'
Line 1
$var!
Line 3
coolicer@ubuntu:~$ echo '$var'!"'$\n'
$var!

```

2.3 为什么cd必须是Shell内建命令

原理：Linux/Unix 的进程模型规定，每个进程只能修改自己的当前工作目录（cwd），子进程无法修改父进程的环境。如果 cd 是独立程序，Shell 会 fork 出一个子进程去执行 cd，子进程目录改了但随即退出，父进程（你的交互式 Shell）目录纹丝不动。因此 cd 必须是内建命令，直接在 Shell 进程内调用 chdir() 系统调用。

验证cd是内建命令：

```

coolicer@ubuntu:~$ type cd
cd is a shell builtin

```

2.4 脚本赋权与执行

原理：Linux 下直接运行 `./script.sh` 需要该文件具备可执行权限（x）。新建的脚本默认没有执行权限，需要通过 `chmod +x` 添加

```
coolicer@ubuntu:~$ echo 'echo "Hello Shell"'>check.sh
coolicer@ubuntu:~$ ./check.sh
bash: ./check.sh: Permission denied
coolicer@ubuntu:~$ chmod +x check.sh
coolicer@ubuntu:~$ ./check.sh
Hello Shell
```

2.5 命令替换与日期备份

原理：`$(command)` 会将括号内命令的执行结果替换到当前位置。结合 `date` 命令格式化输出，可动态生成带日期的文件名。

```
coolicer@ubuntu:~$ echo "test content" > notes.txt
coolicer@ubuntu:~$ cp notes.txt notes_$(date +%Y-%m-%d).txt
coolicer@ubuntu:~$ ls -l notes_*
-rw-rw-r-- 1 coolicer coolicer 13 Aug 31 01:04 notes_2026-08-31.txt
```

2.6 使用awk按列过滤与交换字段

原理：`awk` 默认按空格/制表符分列（`$1`, `$2`, `$3...`）。可以通过条件判断过滤行，并重新赋值列内容后打印。

```
coolicer@ubuntu:~$ printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '($2 > 100 { $1=$3; print })'
y 150 y
z 200 z
```

2.7 克隆本课程网站的仓库

git clone命令来克隆仓库，并转到克隆后的仓库目录下

```
coolicer@ubuntu:~$ git clone https://github.com/missing-semester/missing-semester.git
Cloning into 'missing-semester'...
remote: Enumerating objects: 3455, done.
remote: Counting objects: 100% (224/224), done.
remote: Compressing objects: 100% (134/134), done.
remote: Total 3455 (delta 130), reused 95 (delta 90), pack-reused 3231 (from 4)
Receiving objects: 100% (3455/3455), 12.81 MiB | 54.00 KiB/s, done.
Resolving deltas: 100% (2084/2084), done.
Checking connectivity... done.
coolicer@ubuntu:~$ cd missing-semester
```

2.7.1 可视化版本历史为图

git log命令查看版本历史，-all参数来显示所有分支的提交情况，-graph将路径画成图形

```
coolicer@ubuntu:~/missing-semester$ git log --all --graph --decorate --oneline
* fea9192 (HEAD -> master, origin/master, origin/HEAD) Merge branch 'oiahoon/docs/fix-uv-lock-example'
|
| * 7343431 docs: fix uv lockfile Docker example
|/
* 853e0ea Merge branch 'tufailrizvi-debug/fixing_typo'
|
| * 2df4a16 Fix typos in SSH-related documentation
|/
* adaebe3 Merge branch 'ariuxklc/master'
|
| * de7ac78 Add Mongolian translation link
```

2.7.2 查看最后修改README.md的人

git log查看仓库里的提交历史记录

README.md: 只看跟README.md这个文件有关的提交记录;

-1: 只看其中最近的一条记录

```
coolicer@ubuntu:~/missing-semester$ git log -1 --pretty="最后修改人：%an | 邮箱：%ae" README.md
最后修改人：Anish Athalve | 邮箱：me@anishathalve.com
```

2.8 配置Git别名

目标：让 `git graph` 等价于长命令 `git log --all --graph --decorate --oneline`

```
coolicer@ubuntu:~$ git config --global alias.graph "log --all --graph --decorate --oneline"
coolicer@ubuntu:~$ cd missing-semester
coolicer@ubuntu:~/missing-semester$ git graph
* fea9192 (HEAD -> master, origin/master, origin/HEAD) Merge branch 'oiahoon/docs/fix-uv-lock-example'
|
| * 7343431 docs: fix uv lockfile Docker example
|/
* 853e0ea Merge branch 'tufailrizvi-debug/fixing_typo'
|
| * 2df4a16 Fix typos in SSH-related documentation
|/
```

2.9 配置全局.gitignore

目标：所有仓库自动忽略.DS_Store/编辑器缓存这类不需要提交的文件

```
coolicer@ubuntu:~$ git config --global core.excludesfile ~/.gitignore_global
coolicer@ubuntu:~$ cat > ~/.gitignore_global << 'EOF'
> # macOS垃圾文件
> .DS_Store
> .AppleDouble
> # Windows垃圾
> Thumbs.db
> desktop.ini
> # 编辑器缓存
> .vscode/
> .idea/
> *.swp # Vim临时交换文件
> *~ # Vim自动备份文件
> *.log # 本地日志
> EOF
```

2.10 模拟合并冲突+解决

目标：体验两人改同一行代码的冲突处理流程

`git init`：把当前文件夹初始化成一个空的 Git 仓库(开始用git管理代码)

`cat > recipe.txt && 'EOF' ... EOF`：新建一个叫 `recipe.txt` 的文件，并往里面写入食谱内容（直到遇到 EOF 结束）


```

coolicer@ubuntu:~$ mkdir recipe-demo && cd recipe-demo
coolicer@ubuntu:~/recipe-demo$ git init
Initialized empty Git repository in /home/coolicer/recipe-demo/.git/
coolicer@ubuntu:~/recipe-demo$ cat > recipe.txt << 'EOF'
> 蛋糕食谱：
> 1杯面粉
> 1杯糖
> 2个鸡蛋
> EOF

```

git add . && git commit -m "初始食谱"：把刚才写好的文件交给 Git 暂存，然后正式提交到仓库，提交说明叫“初始食谱”。

git branch salty 创建分支，git checkout salty 切到 salty 分支

```

coolicer@ubuntu:~/recipe-demo$ git add . && git commit -m "初始食谱"
[master (root-commit) 4234b7b] 初始食谱
1 file changed, 4 insertions(+)
create mode 100644 recipe.txt
coolicer@ubuntu:~/recipe-demo$ git branch salty # 咸口需求分支
coolicer@ubuntu:~/recipe-demo$ git branch sweet # 甜口需求分支

```

sed -i 's/1杯糖/2杯糖/' recipe.txt：把文件里的“1杯糖”改成“2杯糖”。

```

coolicer@ubuntu:~/recipe-demo$ git checkout salty
Switched to branch 'salty'
coolicer@ubuntu:~/recipe-demo$ sed -i 's/1杯糖/1杯盐/' recipe.txt
coolicer@ubuntu:~/recipe-demo$ git add . && git commit -m "改成咸口：糖换盐"
[salty 98127ac] 改成咸口：糖换盐
1 file changed, 1 insertion(+), 1 deletion(-)
coolicer@ubuntu:~/recipe-demo$ git checkout sweet
Switched to branch 'sweet'
coolicer@ubuntu:~/recipe-demo$ sed -i 's/1杯糖/2杯糖/' recipe.txt

```

git add . && git commit -m "改成超甜：糖加量为2杯"：修改保存并提交

```

coolicer@ubuntu:~/recipe-demo$ git add . && git commit -m "改成超甜：糖加量为2杯"
[sweet 8844e08] 改成超甜：糖加量为2杯
1 file changed, 1 insertion(+), 1 deletion(-)
coolicer@ubuntu:~/recipe-demo$ git checkout main
error: pathspec 'main' did not match any file(s) known to git.
coolicer@ubuntu:~/recipe-demo$ git merge salty # 第一次合并无冲突，直接成功
Auto-merging recipe.txt
CONFLICT (content): Merge conflict in recipe.txt
Automatic merge failed; fix conflicts and then commit the result.

```

手动重新编辑文件，删除Git生成的冲突标记并提交

```
coolicer@ubuntu:~/recipe-demo$ git merge sweet # 冲突！两个分支都改了「糖」那一行
error: merge is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
coolicer@ubuntu:~/recipe-demo$ nano recipe.txt
coolicer@ubuntu:~/recipe-demo$ git add . && git commit
[sweet da74311] Merge branch 'salty' into sweet
```

以树状图查看刚才的完整历史记录

```
coolicer@ubuntu:~/recipe-demo$ git graph
*   da74311 (HEAD -> sweet) Merge branch 'salty' into sweet
| \
| * 98127ac (salty) 改成咸口：糖换盐
| * | 8844e08 改成超甜：糖加量为2杯
| /
* 4234b7b (master) 初始食谱
```

3 实验心得

通过本次实验，我学会了基本的使用git来进行版本控制的操作方法，以及对如何运用latex进行文档的编辑与排版，感受到了latex编辑与word编辑的不同和它在模版使用方面的便利。